RTL Design Sherpa

RTL AMBA APB5 Module Reference — Rev 0.90

July 2026

Table of Contents

1 APB5 Master (Clock-Gated)

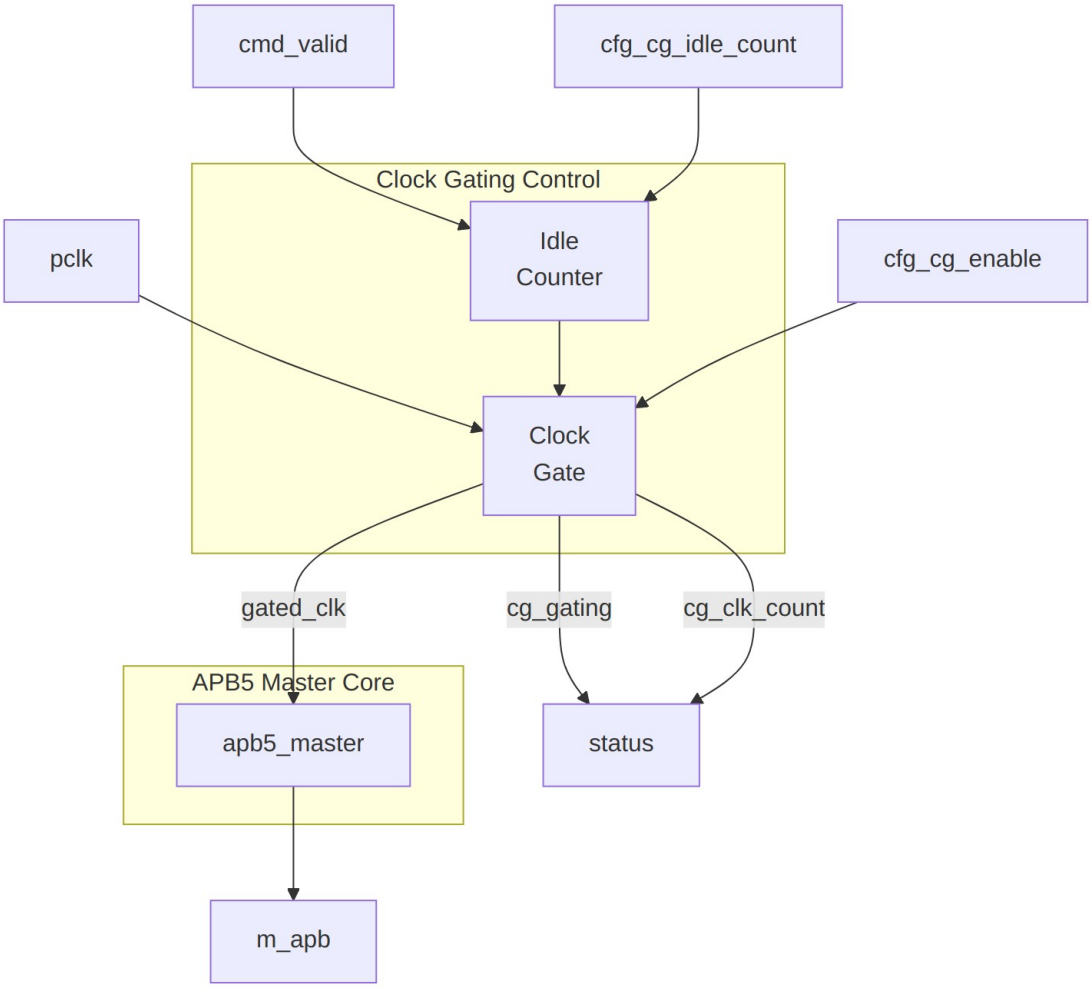
Module: `apb5_master_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

1.1 Overview

Clock-gated variant of the APB5 Master module. Wraps the base `apb5_master` with clock gating control logic to reduce dynamic power consumption during idle periods.

1.1.1 Key Features

- All APB5 Master features (see apb5_master)
- Automatic clock gating during idle periods
- Configurable idle threshold before gating
- Zero-latency wake-up on new transactions
- Power consumption reduction for low-duty-cycle applications



Module Architecture

1.2 Additional Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters inherited from apb5_master.

1.3 Additional Ports

1.3.1 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating (0=disabled)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

1.3.2 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock currently gated
cg_clk_count	32	Output	Cumulative gated clock cycles

1.4 Clock Gating Behavior

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> COUNTING : No activity
    COUNTING --> RUNNING : Activity detected
    COUNTING --> GATED : count >= threshold
    GATED --> RUNNING : cmd_valid or activity

    state RUNNING {
        note right of RUNNING : Clock running
    }
    state COUNTING {
        note right of COUNTING : Counting idle cycles
    }
    state GATED {
        note right of GATED : Clock stopped
    }
```

1.4.1 Timing

```
TODO: Wavedrom timing diagram showing:
- pclk
```

```
- gated_clk
- cmd_valid
- cfg_cg_idle_count
- idle_counter
- cg_gating
- Transaction before/after gating
```

1.5 Usage Example

```
apb5_master_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_master_cg (
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating configuration
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),    // Gate after 8 idle cycles

    // Clock gating status
    .cg_gating          (master_clk_gated),
    .cg_clk_count       (master_gated_cycles),

    // APB5 and command/response interfaces
    // ... (same as apb5_master)
);
```

1.6 Power Savings

Traffic Pattern	Duty Cycle	Typical Savings
Burst	30%	35-40%
Mixed	50%	20-25%
Continuous	90%+	<5%

1.7 Related Documentation

- **APB5 Master** - Base module documentation
- **APB5 Slave CG** - Clock-gated slave
- **Clock Gating Guide** - General clock gating concepts

1.8 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

2 APB5 Master

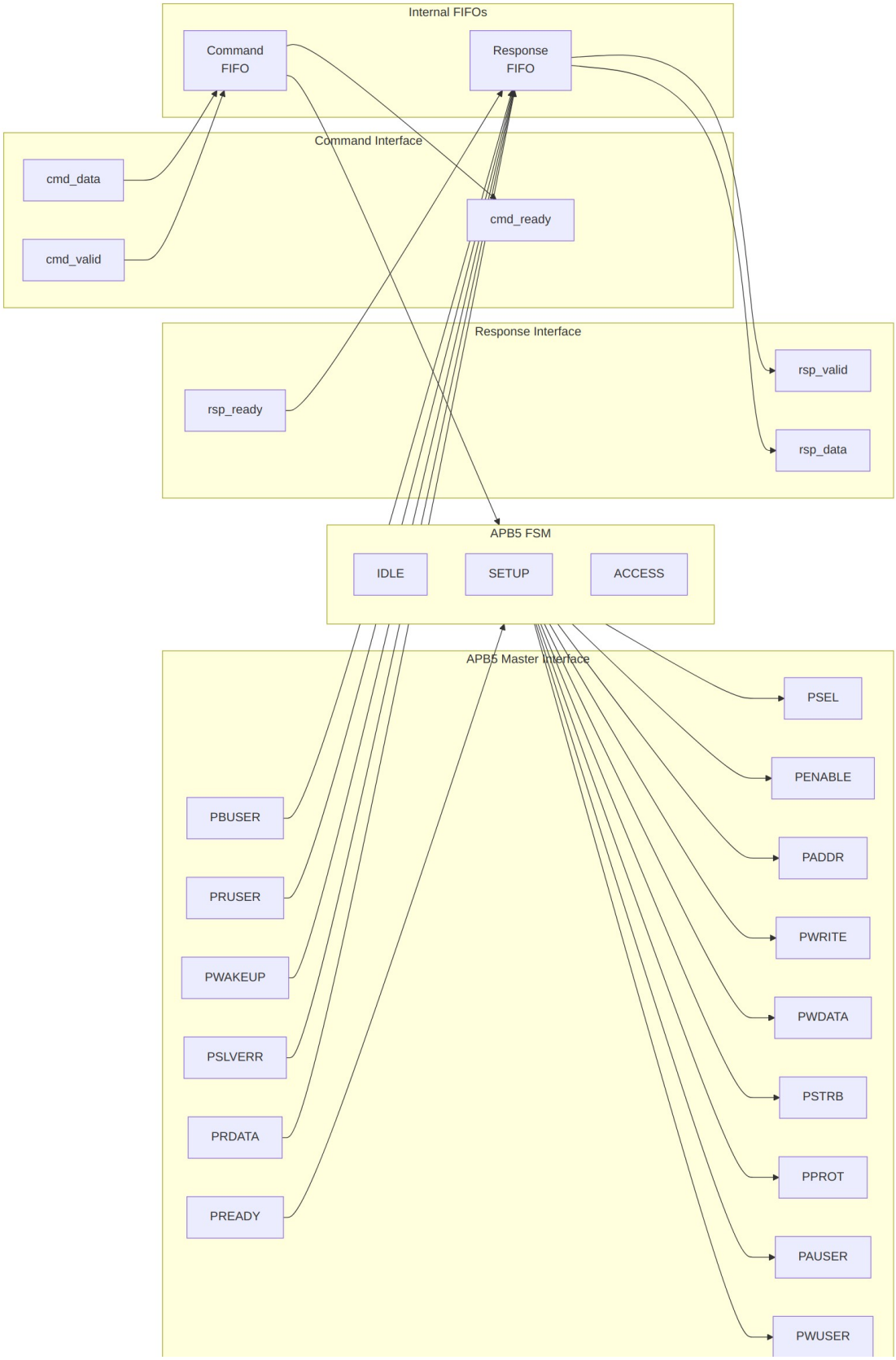
Module: `apb5_master.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

2.1 Overview

The APB5 Master module implements a complete AMBA APB5 master interface with all APB5 extensions including user-defined signals, wake-up support, and optional parity for data integrity. It provides command/response buffering for improved system performance.

2.1.1 Key Features

- Full AMBA APB5 protocol compliance
- PAUSER: User-defined request attributes
- PWUSER: User-defined write data attributes
- PRUSER/PBUSER: User-defined response attributes from slave
- PWAKEUP: Wake-up signal handling from slave
- Optional parity support for data integrity
- Command and response FIFO buffering
- Configurable FIFO depths


Internal FIFOs
Command FIFO
Response FIFO
Command Interface
cmd_data
cmd_valid
cmd_ready
Response Interface
rsp_ready
rsp_valid
rsp_data
APB5 FSM
IDLE
SETUP
ACCESS
APB5 Master Interface
PBUSER
PRUSER
PWAKEUP
PSLVERR
PRDATA
PREADY
PSEL
PENABLE
PADDR
PWRITE
PWDATA
PSTRB
PPROT
PAUSER
PWUSER

2.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/ request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width
CMD_DEPTH	int	6	Command FIFO depth (2^N entries)
RSP_DEPTH	int	6	Response FIFO depth (2^N entries)
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

2.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator (1=write)
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PAUSER	AUSER_WIDTH	Output	User-defined request attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User-defined write data attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PREADY	1	Input	Slave ready

Port	Width	Direction	Description
m_apb_PWAKEUP	1	Input	Wake-up signal from slave
m_apb_PRUSER	RUSER_WIDTH	Input	User-defined read data attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User-defined response attributes

2.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
m_apb_PWDATAPARITY	STRB_WIDTH	Output	Write data parity (per byte)
m_apb_PADDRPARITY	1	Output	Address parity
m_apb_PCTRLPARITY	1	Output	Control signals parity
m_apb_PRDATAPARITY	STRB_WIDTH	Input	Read data parity from slave
m_apb_PREADYPARITY	1	Input	PREADY parity from slave
m_apb_PSLVERRPARITY	1	Input	PSLVERR parity from slave

2.3.4 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO

Port	Width	Direction	Description
			not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection attributes
cmd_pauser	AUSER_WIDTH	Input	Command user attributes
cmd_pwuser	WUSER_WIDTH	Input	Command write user attributes

2.3.5 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data
rsp_pslverr	1	Output	Response error status
rsp_pwakeup	1	Output	Response wake-up indicator
rsp_pruser	RUSER_WIDTH	Output	Response read user attributes
rsp_pbuser	BUSER_WIDTH	Output	Response user

Port	Width	Direction	Description
			attributes

2.4 Functionality

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> SETUP : cmd_fifo_valid
    SETUP --> ACCESS : always
    ACCESS --> IDLE : PREADY & cmd_fifo_empty
    ACCESS --> SETUP : PREADY & !cmd_fifo_empty

    state IDLE {
        note right of IDLE : PSEL=0, PENABLE=0
    }
    state SETUP {
        note right of SETUP : PSEL=1, PENABLE=0
    }
    state ACCESS {
        note right of ACCESS : PSEL=1, PENABLE=1
    }
```

2.4.1 APB5 Extensions

User Signals: - **PAUSER**: Carries user-defined attributes with the address/control phase - **PWUSER**: Carries user-defined attributes with write data - **PRUSER**: Returns user-defined attributes with read data - **PBUSER**: Returns user-defined attributes with the response

Wake-up Support: - **PWAKEUP**: Slave can assert to indicate wake-up events - Captured in response packet for software handling

Parity Protection: - Optional odd parity on data, address, and control signals - Enables detection of single-bit transmission errors

---

2.5 Timing Diagrams

2.5.1 Basic Write Transaction

```
TODO: Wavedrom timing diagram showing:
- PCLK
- PSEL
- PENABLE
- PADDR
```

- PWRITE (high)
- PWDATA
- PSTRB
- PAUSER
- PWUSER
- PREADY
- PSLVERR

2.5.2 Basic Read Transaction

TODO: Wavedrom timing diagram showing:
- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER
- PSLVERR

2.5.3 Wake-up Signal Handling

TODO: Wavedrom timing diagram showing:
- PCLK
- Transaction signals
- PWAKEUP assertion
- Response capture

2.6 Usage Example

```
apb5_master #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .CMD_DEPTH      (4),
    .RSP_DEPTH      (4),
    .ENABLE_PARITY  (0)
) u_apb5_master (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 master interface
```

```
    .m_apb_PSEL     (m_apb_psel),
    .m_apb_PENABLE  (m_apb_penable),
    .m_apb_PADDR    (m_apb_paddr),
    .m_apb_PWRITE   (m_apb_pwrite),
    .m_apb_PWDATA   (m_apb_pwdata),
    .m_apb_PSTRB    (m_apb_pstrb),
    .m_apb_PPROT    (m_apb_pprot),
    .m_apb_PAUSER   (m_apb_pauser),
    .m_apb_PWUSER   (m_apb_pwuser),
    .m_apb_PRDATA   (m_apb_prdata),
    .m_apb_PSLVERR  (m_apb_pslverr),
    .m_apb_PREADY   (m_apb_pready),
    .m_apb_PWAKEUP  (m_apb_pwakeup),
    .m_apb_PRUSER   (m_apb_pruser),
    .m_apb_PBUSER   (m_apb_pbuser),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_write),
    .cmd_paddr      (cmd_addr),
    .cmd_pwdata     (cmd_wdata),
    .cmd_pstrb      (cmd_strb),
    .cmd_pprot      (cmd_prot),
    .cmd_pauser     (cmd_auser),
    .cmd_pwuser     (cmd_wuser),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_rdata),
    .rsp_pslverr    (rsp_error),
    .rsp_pwakeup    (rsp_wakeup),
    .rsp_pruser     (rsp_ruser),
    .rsp_pbuser     (rsp_buser)
);
```

2.7 Design Notes

2.7.1 APB5 vs APB4 Differences

Feature	APB4	APB5
User signals	None	PAUSER, PWUSER, PRUSER, PBUSER

Feature	APB4	APB5
Wake-up	None	PWAKEUP
Parity	None	Optional on all signals

2.7.2 FIFO Sizing

- Command FIFO depth should match expected command burst length
- Response FIFO depth should match to prevent backpressure
- Typical values: 4-16 entries (depth parameter is log2)

2.7.3 Parity Implementation

When `ENABLE_PARITY=1`: - Odd parity computed on outgoing signals - Incoming parity checked (error handling is system-specific) - Adds latency for parity computation

---

2.8 Related Documentation

- **APB5 Slave** - APB5 slave interface
- **APB5 Master CG** - Clock-gated variant
- **APB5 Monitor** - Protocol monitor
- **APB4 Master** - APB4 version for comparison

2.9 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

3 APB5 Master Stub

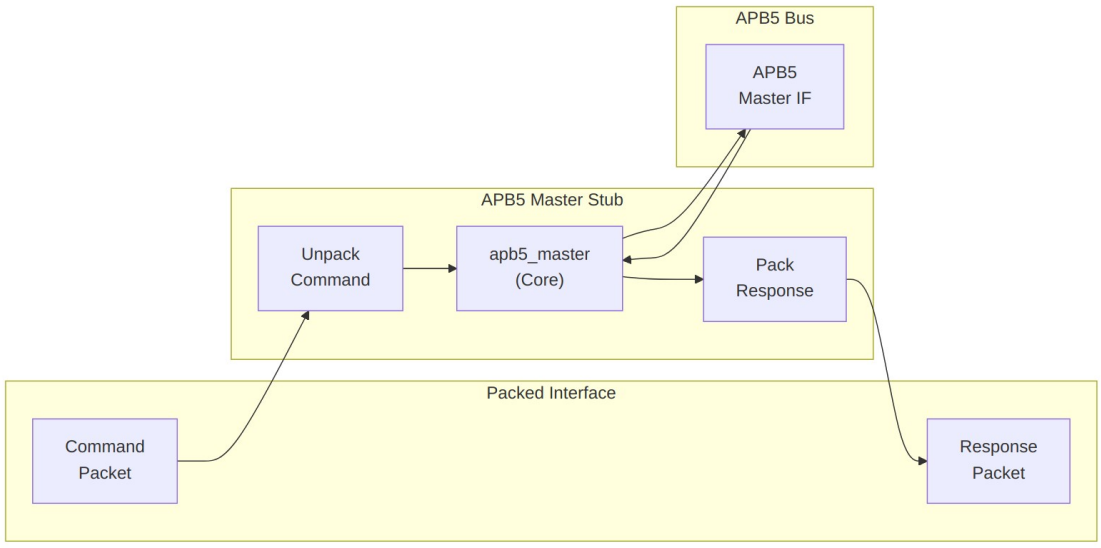
Module: `apb5_master_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

3.1 Overview

The APB5 Master Stub provides a simplified packed-data interface for driving APB5 transactions. It wraps the full `apb5_master` module with packet-based command/response interfaces, making it ideal for testbenches and integration scenarios where a simpler interface is preferred.

3.1.1 Key Features

- Packed command/response packet interface
- Wraps fully-tested `apb5_master` internally
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal handling from slave
- Optional parity support
- First/last markers for transaction tracking



Module Architecture

3.2 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command FIFO depth (log2)
RSP_DEPTH	int	6	Response FIFO

Parameter	Type	Default	Description
			depth (log2)
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

3.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PAUSER	AUSER_WIDTH	Output	User request attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User write attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PREADY	1	Input	Slave ready
m_apb_PWAKEUP	1	Input	Wake-up from slave
m_apb_PRUSER	RUSER_WIDTH	Input	User read attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User response attributes

3.3.3 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready to accept command
cmd_data	CMD_PACKET_WIDTH	Input	Packed command data

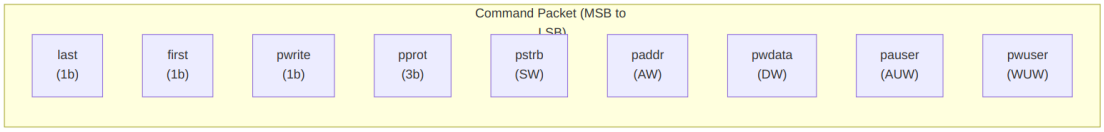
3.3.4 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready to accept response
rsp_data	RESP_PACKET_WIDTH	Output	Packed response data

3.3.5 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	Read data parity error
parity_error_ctrl	1	Output	Control signal parity error
wakeup_pending	1	Output	Wake-up signal active

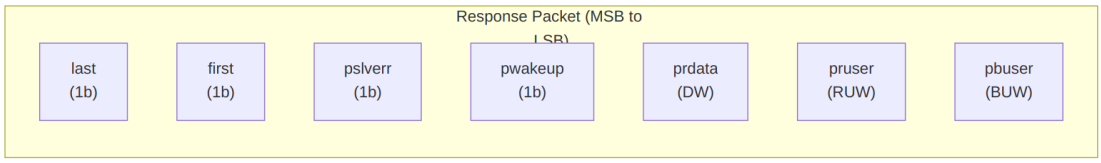
3.4 Packet Formats



Command Packet Structure

Bit Positions:

```
cmd_data = {last, first, pwrite, pprot, pstrb, paddr, pwdata, pauser,
pwuser}
```

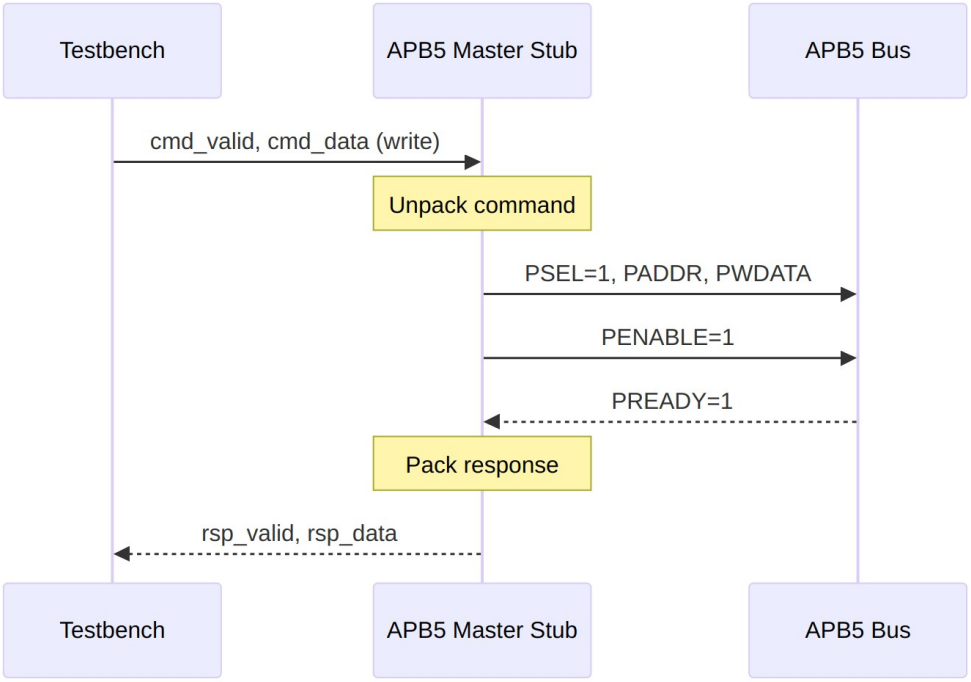


Response Packet Structure

Bit Positions:

```
rsp_data = {last, first, pslverr, pwakeup, prdata, pruser, pbuser}
```

3.5 Transaction Flow



Write Transaction

3.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- pclk
- cmd_valid, cmd_ready, cmd_data
- APB signals (PSEL, PENABLE, PADDR, PWDATA, PREADY)
- rsp_valid, rsp_ready, rsp_data
- Packet-to-APB timing relationship
```

3.6 Usage Example

```
apb5_master_stub #(
    .CMD_DEPTH      (6),
    .RSP_DEPTH      (6),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
```

```
) u_apb5_master_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 master interface
    .m_apb_PSEL     (m_apb_psel),
    .m_apb_PENABLE  (m_apb_penable),
    .m_apb_PADDR    (m_apb_paddr),
    .m_apb_PWRITE   (m_apb_pwrite),
    .m_apb_PWDATA   (m_apb_pwdata),
    .m_apb_PSTRB    (m_apb_pstrb),
    .m_apb_PPROT    (m_apb_pprot),
    .m_apb_PAUSER   (m_apb_pauser),
    .m_apb_PWUSER   (m_apb_pwuser),
    .m_apb_PRDATA   (m_apb_prdata),
    .m_apb_PSLVERR  (m_apb_pslverr),
    .m_apb_PREADY   (m_apb_pready),
    .m_apb_PWAKEUP  (m_apb_pwakeup),
    .m_apb_PRUSER   (m_apb_pruser),
    .m_apb_PBUSER   (m_apb_pbuser),

    // Packed command interface
    .cmd_valid      (tb_cmd_valid),
    .cmd_ready      (tb_cmd_ready),
    .cmd_data       (tb_cmd_data),

    // Packed response interface
    .rsp_valid      (tb_rsp_valid),
    .rsp_ready      (tb_rsp_ready),
    .rsp_data       (tb_rsp_data),

    // Status
    .wakeup_pending (apb_wakeup)
);

// Build command packet
assign tb_cmd_data = {
    1'b1,           // last
    1'b1,           // first
    1'b1,           // pwrite (write transaction)
    3'b000,         // pprot
    4'hF,           // pstrb (all bytes)
    32'h1000_0000,  // paddr
    32'hDEAD_BEEF,  // pwdata
    4'h0,           // pauser
    4'h0            // pwuser
};
```

3.7 Design Notes

3.7.1 First/Last Markers

The first/last bits in packets enable: - Transaction boundary detection - Burst transaction support - Testbench synchronization

3.7.2 Internal Architecture

The stub instantiates the full `apb5_master` internally: - All protocol handling done by proven `apb5_master` - Stub only handles packet packing/unpacking - Maintains full APB5 compliance

3.8 Related Documentation

- **APB5 Master** - Core master module (wrapped by stub)
- **APB5 Slave Stub** - Corresponding slave stub

3.9 Navigation

- **<- Back to APB5 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

4 APB5 Slave (CDC + Clock-Gated)

Module: `apb5_slave_cdc_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

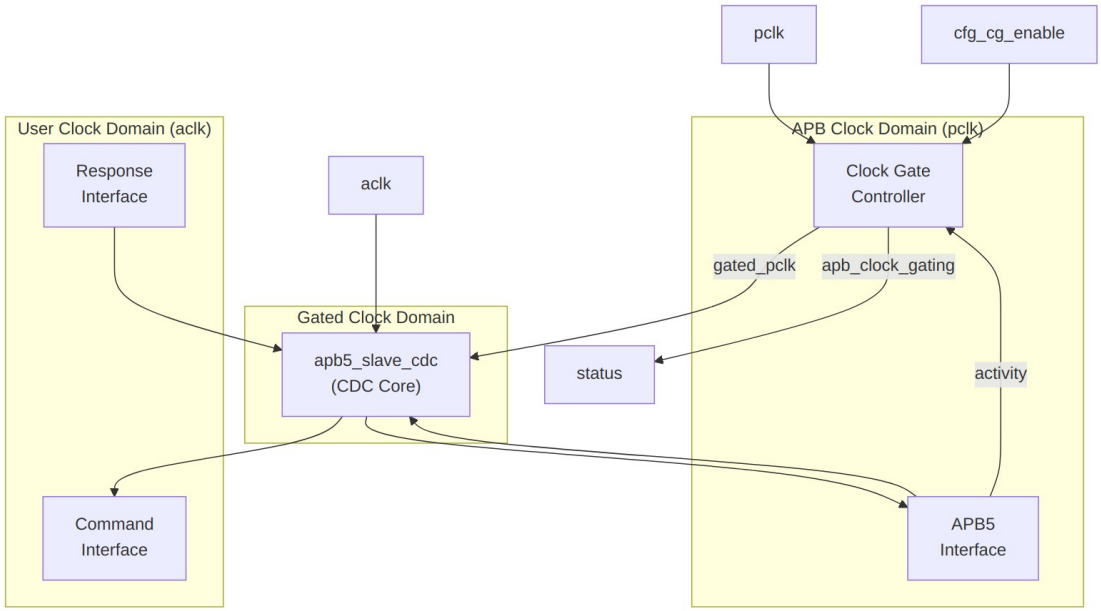
4.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps `apb5_slave_cdc` with clock gating control to reduce power consumption during idle periods while maintaining safe operation across asynchronous clock boundaries.

4.1.1 Key Features

- Full APB5 protocol support with all extensions

- Asynchronous clock domain crossing (APB to backend)
- Clock gating for power reduction during idle
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal handling across domains
- Optional parity support for data integrity
- Automatic wake-up on transaction activity



Module Architecture

4.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width

Parameter	Type	Default	Description
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Internal buffer depth
ENABLE_PARITY	bit	0	Enable parity signals
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

4.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

4.3.3 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain.

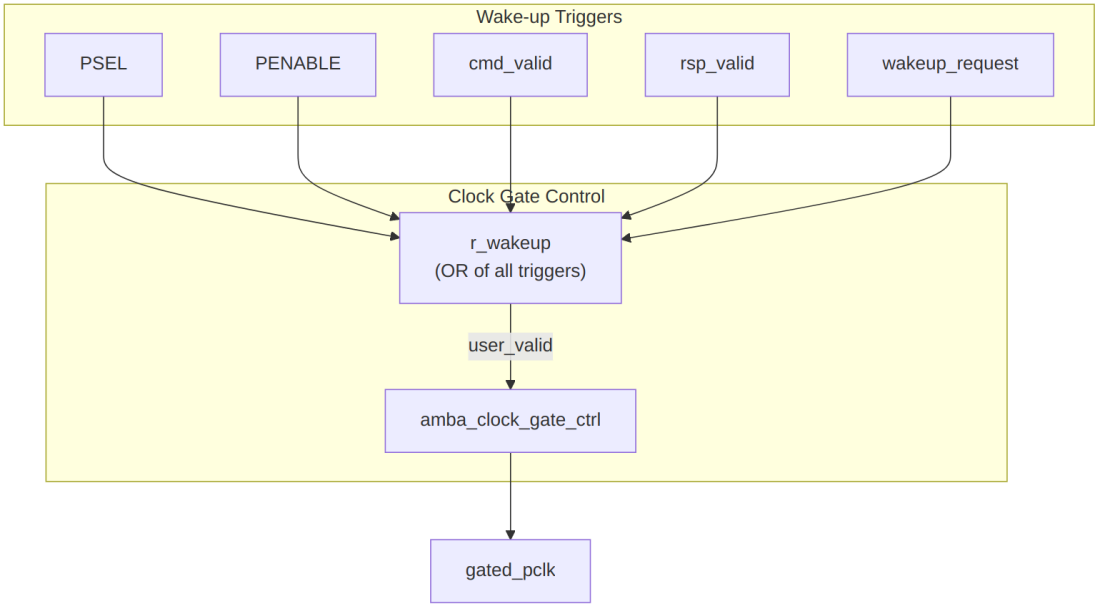
4.3.4 Backend Interface

Same command/response interface as apb5_slave_cdc - operates in `aclk` domain.

4.3.5 Status Outputs

Port	Width	Direction	Description
apb_clock_gating	1	Output	Indicates clock is currently gated
parity_error_wdata	1	Output	Write data parity error detected
parity_error_ctrl	1	Output	Control signal parity error

4.4 Clock Gating and CDC Interaction



Wake-up Logic

4.4.1 Timing Considerations

TODO: Wavedrom timing diagram showing:
- pclk (ungated)
- gated_pclk
- aclk (different frequency)
- s_apb_PSEL (wake trigger)
- apb_clock_gating indicator
- Transaction flow across CDC with gating
- Wake-up latency from PSEL to clock active

4.5 Usage Example

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .DEPTH              (2),
    .ENABLE_PARITY      (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // User clock domain
    .aclk               (user_clk),
    .aresetn            (user_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),
    .apb_clock_gating   (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL         (s_apb_psel),
    .s_apb_PENABLE      (s_apb_penable),
    .s_apb_PREADY       (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid          (backend_cmd_valid),
    .cmd_ready          (backend_cmd_ready),
```

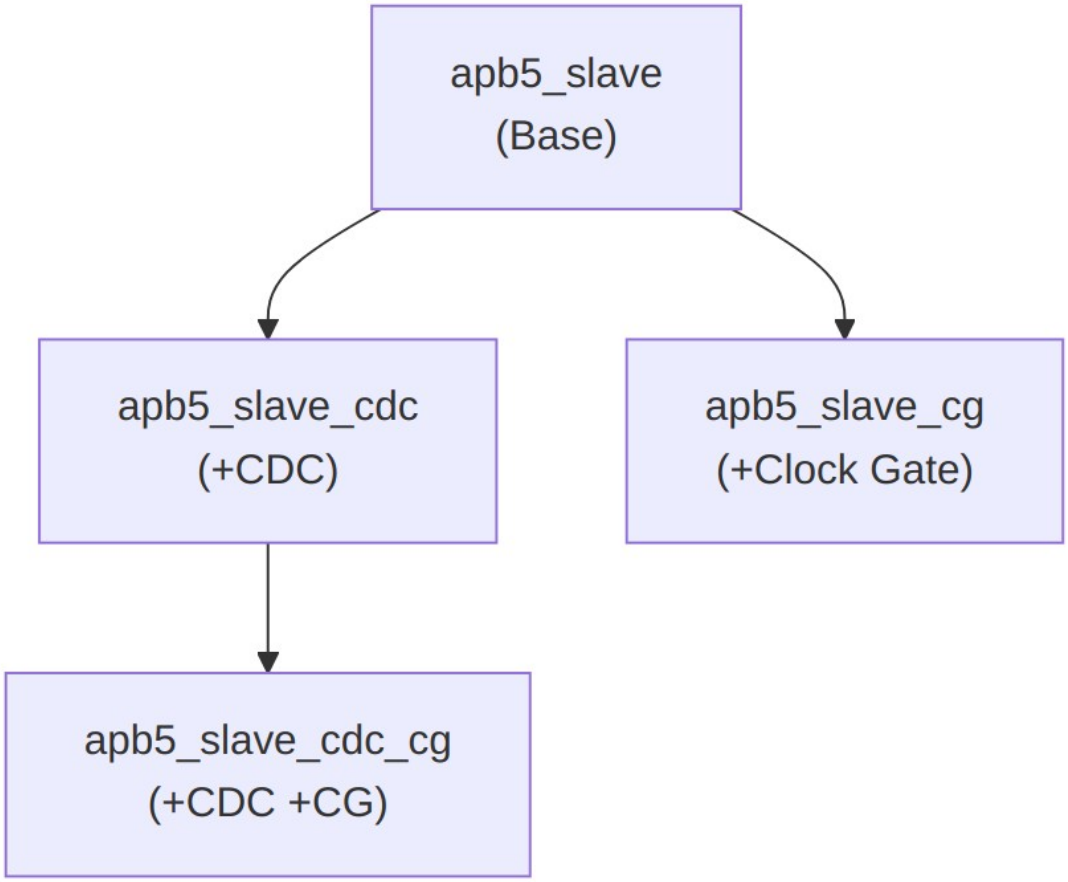
```
    // ... other command signals

    .rsp_valid          (backend_rsp_valid),
    .rsp_ready          (backend_rsp_ready),
    // ... other response signals

    // Wake-up control (aclk domain)
    .wakeup_request     (backend_wakeup)
);
```

4.6 Design Notes

4.6.1 Power and Latency Trade-offs

Configuration	Power Savings	Wake-up Latency
CG disabled	None	0 cycles
CG idle=4	Moderate	0 cycles (instant wake)
CG idle=16	Good	0 cycles (instant wake)

Note: Wake-up is instant from PSEL assertion due to combinational wake-up logic.



Combined Feature Hierarchy

4.6.2 Reset Considerations

- APB domain reset (`presetn`) controls APB interface and clock gate
- User domain reset (`aresetn`) controls backend interface
- Both resets must be properly synchronized to their domains
- Module handles internal CDC for reset coordination

4.7 Related Documentation

- **APB5 Slave** - Base slave module
- **APB5 Slave CDC** - CDC variant without clock gating
- **APB5 Slave CG** - Clock gating without CDC

4.8 Navigation

- <- Back to APB5 Index
- <- Back to RTLAmba Index
- <- Back to Main Documentation Index

5 APB5 Slave (Clock Domain Crossing)

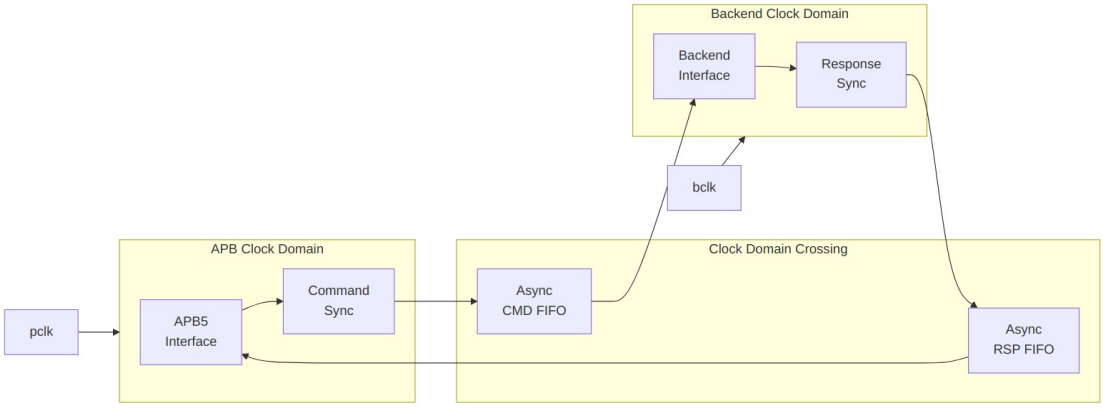
Module: `apb5_slave_cdc.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

5.1 Overview

The APB5 Slave CDC module provides clock domain crossing capability between an APB5 bus clock domain and a backend clock domain. It safely transfers APB5 transactions across asynchronous clock boundaries.

5.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal crossing
- Configurable FIFO depths for each direction
- Metastability protection



Module Architecture

5.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
CMD_DEPTH	int	4	Command FIFO depth (power of 2)
RSP_DEPTH	int	4	Response FIFO depth (power of 2)
SYNC_STAGES	int	2	Synchronizer stages for CDC

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
bclk	1	Input	Backend clock

Port	Width	Direction	Description
bresetn	1	Input	Backend reset (active low)

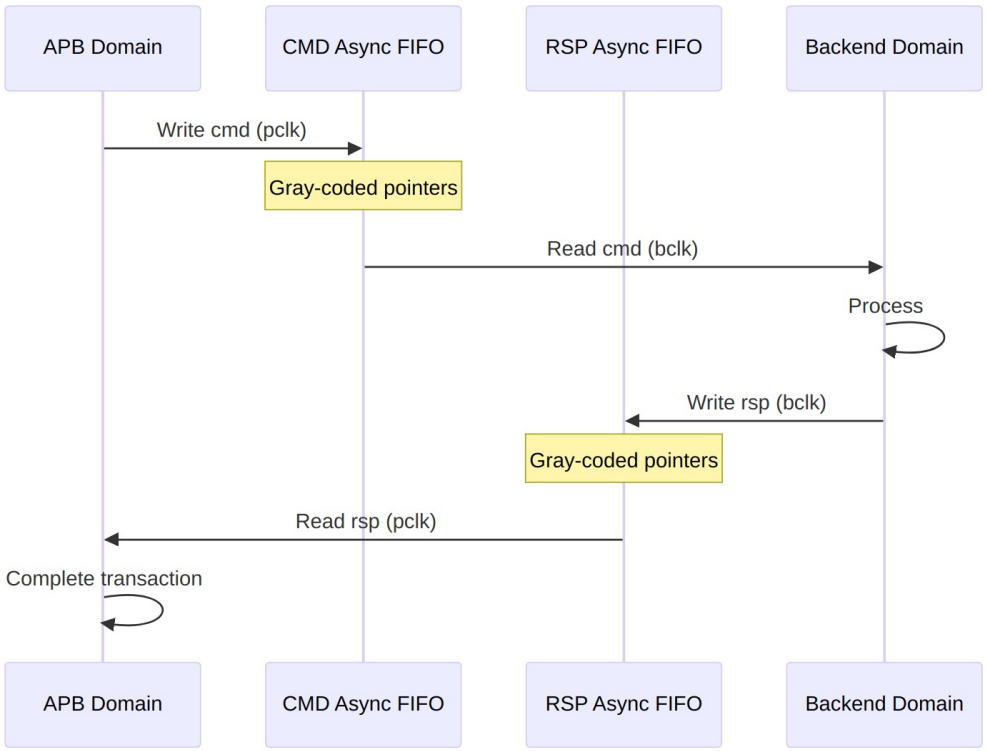
5.3.2 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain.

5.3.3 Backend Interface

Same command/response interface as apb5_slave - operates in `bclk` domain.

5.4 Clock Domain Crossing



CDC Mechanism

5.4.1 Timing Considerations

```
TODO: Wavedrom timing diagram showing:
- pclk
- bclk (different frequency)
- APB transaction signals
```

```
- FIFO write/read pointers
- Backend transaction signals
- CDC latency
```

5.5 Usage Example

```
apb5_slave_cdc #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .CMD_DEPTH      (4),
    .RSP_DEPTH      (4),
    .SYNC_STAGES    (2)
) u_apb5_slave_cdc (
    // APB clock domain
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // Backend clock domain
    .bclk           (backend_clk),
    .bresetn        (backend_rst_n),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    // ... other APB signals

    // Backend interface (bclk domain)
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    // ... other backend signals
);
```

5.6 Design Notes

5.6.1 FIFO Depth Sizing

- CMD_DEPTH should handle worst-case APB burst before backend responds
- RSP_DEPTH should handle responses during slow APB clock periods

- Minimum depth: 2 (for proper CDC operation)

5.6.2 Reset Synchronization

- Both resets should be properly synchronized to their respective domains
- Module handles internal reset synchronization for CDC logic

5.6.3 Latency

- Minimum latency: 2-4 cycles per direction (sync stages)
- Additional latency if FIFOs are empty/full

5.7 Related Documentation

- **APB5 Slave** - Base slave without CDC
- **APB5 Slave CDC CG** - CDC with clock gating
- **CDC Handshake** - CDC design patterns

5.8 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

6 APB5 Slave (Clock-Gated)

Module: `apb5_slave_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

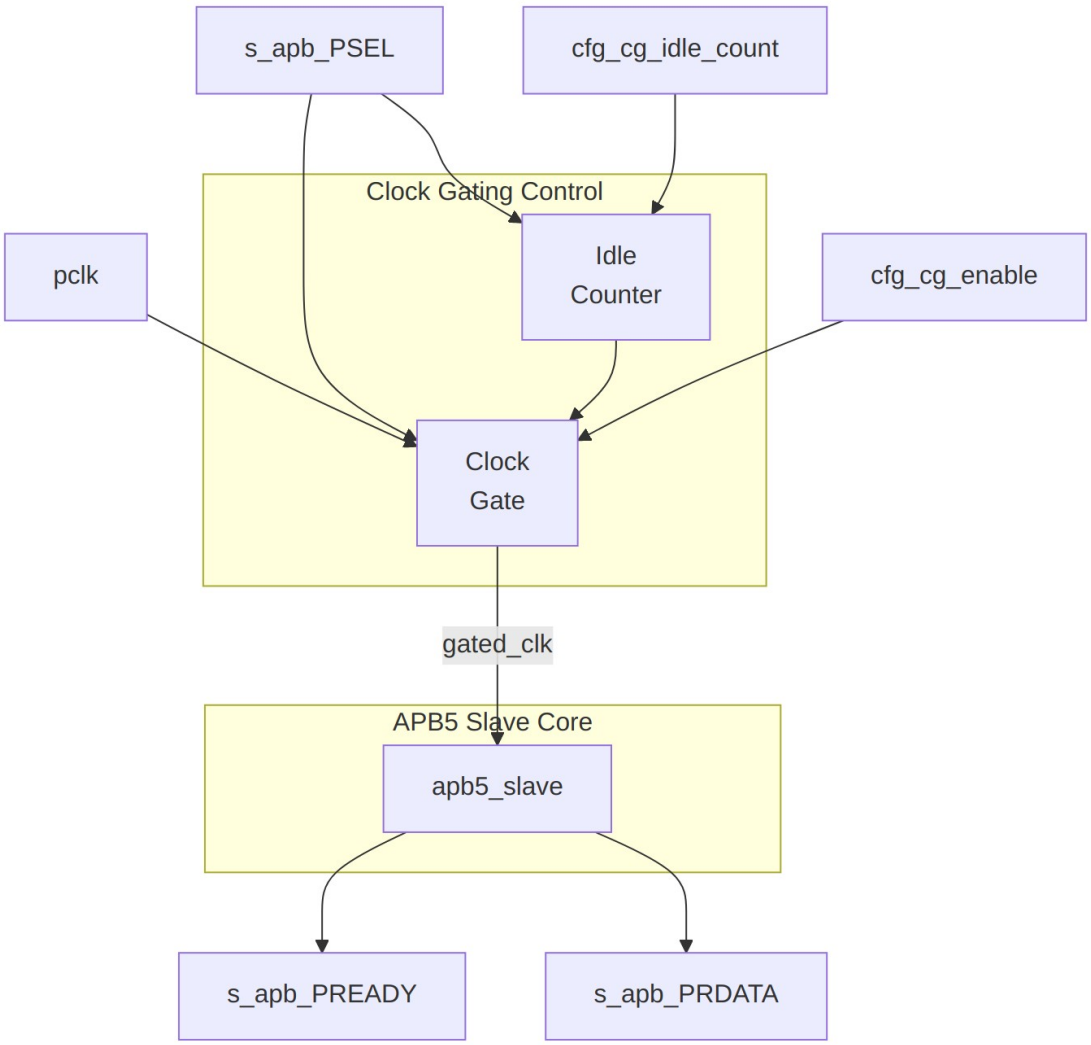
6.1 Overview

Clock-gated variant of the APB5 Slave module. Wraps the base `apb5_slave` with clock gating control logic to reduce dynamic power consumption when no transactions are active.

6.1.1 Key Features

- All APB5 Slave features (see apb5_slave)
- Automatic clock gating during idle periods
- Immediate wake-up on PSEL assertion
- Configurable idle threshold

- Power monitoring outputs



Module Architecture

6.2 Additional Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter

All other parameters inherited from apb5_slave.

6.3 Additional Ports

6.3.1 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

6.3.2 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock currently gated
cg_clk_count	32	Output	Cumulative gated cycles

6.4 Clock Gating Behavior

6.4.1 Wake-up Trigger

The slave clock ungates immediately when: - PSEL is asserted (new transaction starting) - Configuration changes

```
TODO: Wavedrom timing diagram showing:
- pclk
- gated_clk
- s_apb_PSEL
- cg_gating
- Wake-up latency (0 cycles)
```

6.5 Usage Example

```
apb5_slave_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cg (
```

```
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd4),
    .cg_gating          (slave_clk_gated),
    .cg_clk_count       (slave_gated_cycles),

    // APB5 interface (same as apb5_slave)
    // ...
);
```

6.6 Related Documentation

- **APB5 Slave** - Base module documentation
- **APB5 Master CG** - Clock-gated master

6.7 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

7 APB5 Slave

Module: `apb5_slave.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

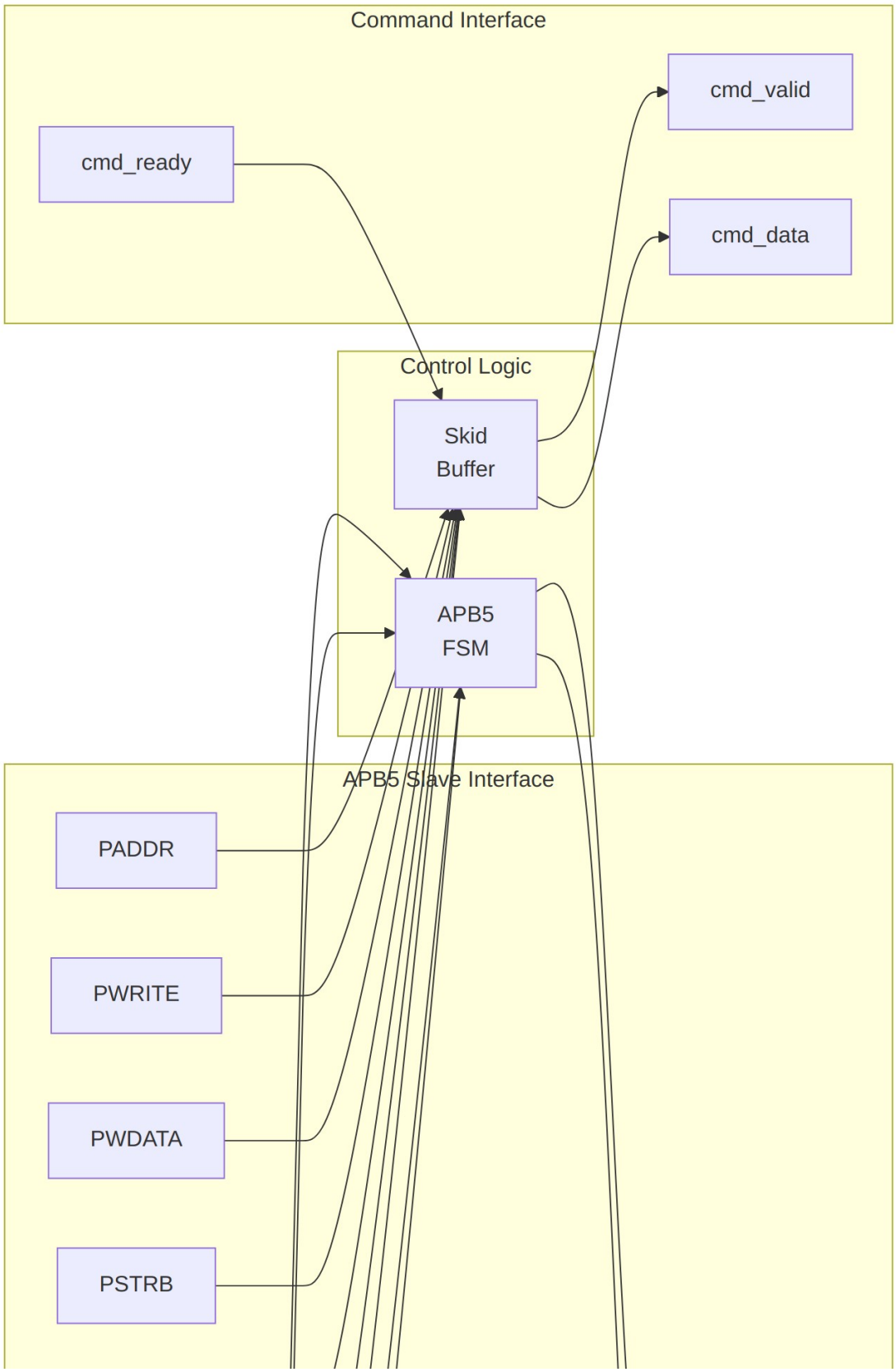
7.1 Overview

The APB5 Slave module implements a complete AMBA APB5 slave interface with all APB5 extensions. It receives APB transactions from the bus and provides a command/response interface for backend logic, supporting user-defined signals, wake-up generation, and optional parity.

7.1.1 Key Features

- Full AMBA APB5 protocol compliance
- Receives PAUSER/PWUSER from master
- Generates PRUSER/PBUSER responses

- PWAKEUP output for wake-up signaling
- Optional parity support for data integrity
- Command/response FIFO buffering
- Configurable buffer depth


Command Interface
cmd_valid
cmd_ready
cmd_data
Control Logic
Skid
Buffer
APB5
FSM
APB5 Slave Interface
PADDR
PWRITE
PWDATA
PSTRB

7.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/ request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Internal buffer depth
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock

Port	Width	Direction	Description
presetn	1	Input	APB active-low reset

7.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	Slave ready response
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User-defined request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User-defined write data attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PWAKEUP	1	Output	Wake-up signal to master
s_apb_PRU	RUSER_WIDT	Output	User-defined read data

Port	Width	Direction	Description
SER	H		attributes
s_apb_PBUSER	BUSER_WIDTH	Output	User-defined response attributes

7.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
s_apb_PWDATAPARITY	STRB_WIDTH	Input	Write data parity from master
s_apb_PADDRPARITY	1	Input	Address parity from master
s_apb_PCTRLPARITY	1	Input	Control signals parity from master
s_apb_PRDATAPARITY	STRB_WIDTH	Output	Read data parity to master
s_apb_PREADYPARITY	1	Output	PREADY parity to master
s_apb_PSLVERRPARITY	1	Output	PSLVERR parity to master

7.3.4 Command Interface (to backend)

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid to backend
cmd_ready	1	Input	Backend ready
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command

Port	Width	Direction	Description
			write data
cmd_pstrb	STRB_WIDTH	Output	Command write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection
cmd_pauser	AUSER_WIDTH	Output	Command address user
cmd_pwuser	WUSER_WIDTH	Output	Command write user

7.3.5 Response Interface (from backend)

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid from backend
rsp_ready	1	Output	Slave ready for response
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error status
rsp_pruser	RUSER_WIDTH	Input	Response read user
rsp_pbuser	BUSER_WIDTH	Input	Response user

7.3.6 Wake-up Control

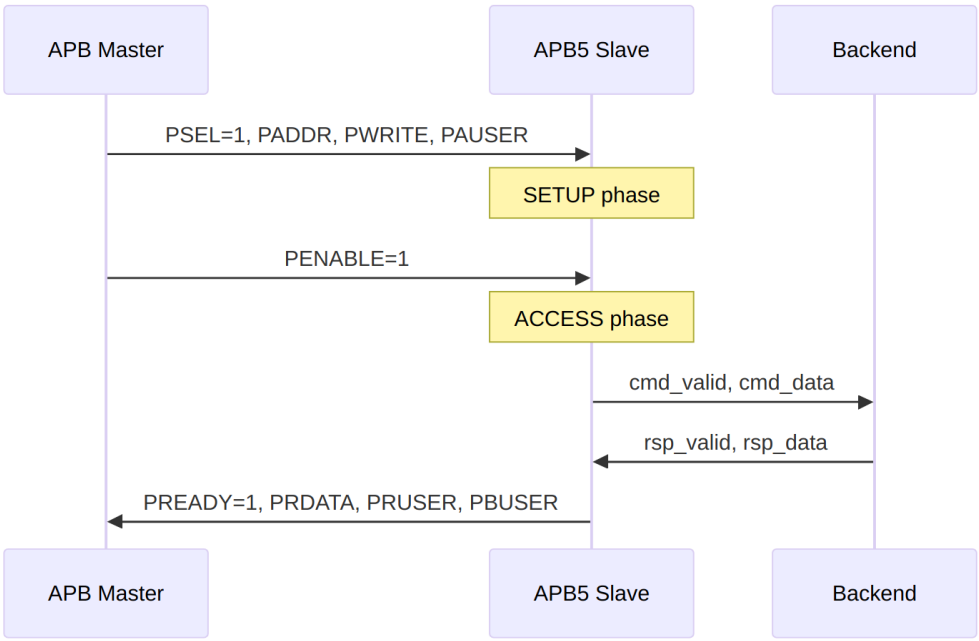
Port	Width	Direction	Description
wakeup_req	1	Input	Wake-up request from backend

7.4 Functionality

7.4.1 APB5 Slave Protocol

The slave responds to APB transactions with proper timing:

1. **SETUP Phase**: PSEL=1, PENABLE=0 - Capture address and control
2. **ACCESS Phase**: PSEL=1, PENABLE=1 - Assert PREADY when response ready



Response Timing

7.5 Timing Diagrams

7.5.1 Write Transaction with Wait States

```
TODO: Wavedrom timing diagram showing:
- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (high)
- PWDATA
- PAUSER, PWUSER
- PREADY (with wait states)
- PSLVERR
- PBUSER
```

7.5.2 Read Transaction

TODO: Wavedrom timing diagram showing:
- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER, PBUSER

7.5.3 Wake-up Signaling

TODO: Wavedrom timing diagram showing:
- PCLK
- wakeup_req (from backend)
- PWAKEUP (to master)
- Timing relationship

7.6 Usage Example

```
apb5_slave #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PREADY   (s_apb_pready),
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
```

```
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PWAKEUP  (s_apb_pwakeup),
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Backend command interface
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    .cmd_pwrite     (backend_cmd_write),
    .cmd_paddr      (backend_cmd_addr),
    .cmd_pwdata     (backend_cmd_wdata),
    .cmd_pstrb      (backend_cmd_strb),
    .cmd_pprot      (backend_cmd_prot),
    .cmd_pauser     (backend_cmd_auser),
    .cmd_pwuser     (backend_cmd_wuser),

    // Backend response interface
    .rsp_valid      (backend_rsp_valid),
    .rsp_ready      (backend_rsp_ready),
    .rsp_prdata     (backend_rsp_rdata),
    .rsp_pslverr    (backend_rsp_error),
    .rsp_pruser     (backend_rsp_ruser),
    .rsp_pbuser     (backend_rsp_buser),

    // Wake-up
    .wakeup_req     (backend_wakeup)
);
```

7.7 Design Notes

7.7.1 Backpressure Handling

- If backend cannot accept commands (`cmd_ready=0`), slave inserts wait states
- PREADY held low until backend accepts command and provides response

7.7.2 User Signal Propagation

- PAUSER/PWUSER captured during SETUP phase, forwarded to backend
- PRUSER/PBUSER from backend driven during ACCESS phase response

7.8 Related Documentation

- **APB5 Master** - APB5 master interface
- **APB5 Slave CG** - Clock-gated variant
- **APB5 Slave CDC** - Clock domain crossing variant
- **APB4 Slave** - APB4 version for comparison

7.9 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

8 APB5 Slave Stub

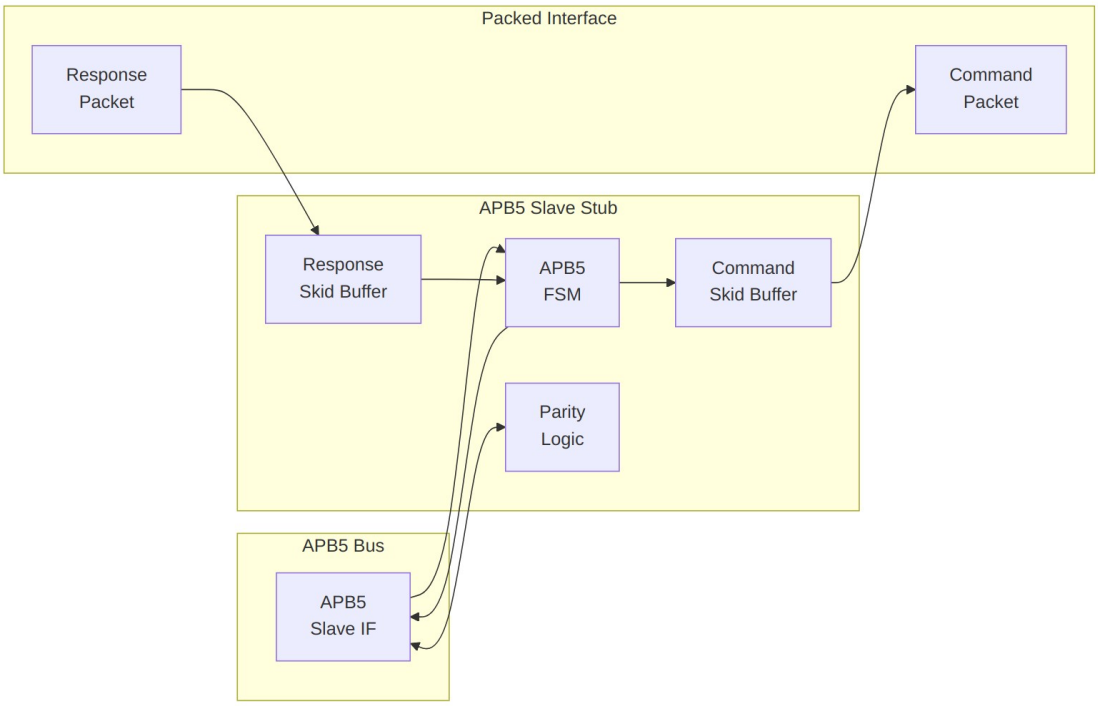
Module: `apb5_slave_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

8.1 Overview

The APB5 Slave Stub provides a simplified packed-data interface for responding to APB5 transactions. It implements a complete APB5 slave with packet-based command/response interfaces, making it ideal for testbenches and simple backend implementations.

8.1.1 Key Features

- Packed command/response packet interface
- Complete APB5 slave FSM implementation
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal generation to master
- Optional parity support with error detection
- Skid buffers for command and response paths



Module Architecture

8.2 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Skid buffer depth
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user

Parameter	Type	Default	Description
			signal width
BUSER_WIDTH	int	4	Response user signal width
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

8.3 Ports

8.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

8.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte

Port	Width	Direction	Description
			strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User write attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PREADY	1	Output	Slave ready
s_apb_PWAKEUP	1	Output	Wake-up to master
s_apb_PRUSER	RUSER_WIDTH	Output	User read attributes
s_apb_PBUSER	BUSER_WIDTH	Output	User response attributes

8.3.3 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Backend ready for command
cmd_data	CMD_PACKET_WIDTH	Output	Packed command data

8.3.4 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for response

Port	Width	Direction	Description
rsp_data	RESP_PACKET_WIDTH	Input	Packed response data

8.3.5 Control and Status

Port	Width	Direction	Description
wakeup_request	1	Input	Assert PWAKEUP to master
parity_error_wdata	1	Output	Write data parity error
parity_error_ctrl	1	Output	Control signal parity error

8.4 Packet Formats

8.4.1 Command Packet Structure

```
cmd_data = {pwrite, pprot, pstrb, paddr, pwdata, pauser, pwuser}
```

Field	Width	Description
pwrite	1	Write/read indicator
pprot	PROT_WIDTH	Protection attributes
pstrb	STRB_WIDTH	Write byte strobes
paddr	ADDR_WIDTH	Transaction address
pwdata	DATA_WIDTH	Write data
pauser	AUSER_WIDTH	Address user signal
pwuser	WUSER_WIDTH	Write user signal

8.4.2 Response Packet Structure

```
rsp_data = {pslverr, prdata, pruser, pbuser}
```

Field	Width	Description
pslverr	1	Error response
prdata	DATA_WIDTH	Read data
pruser	RUSER_WIDTH	Read user signal

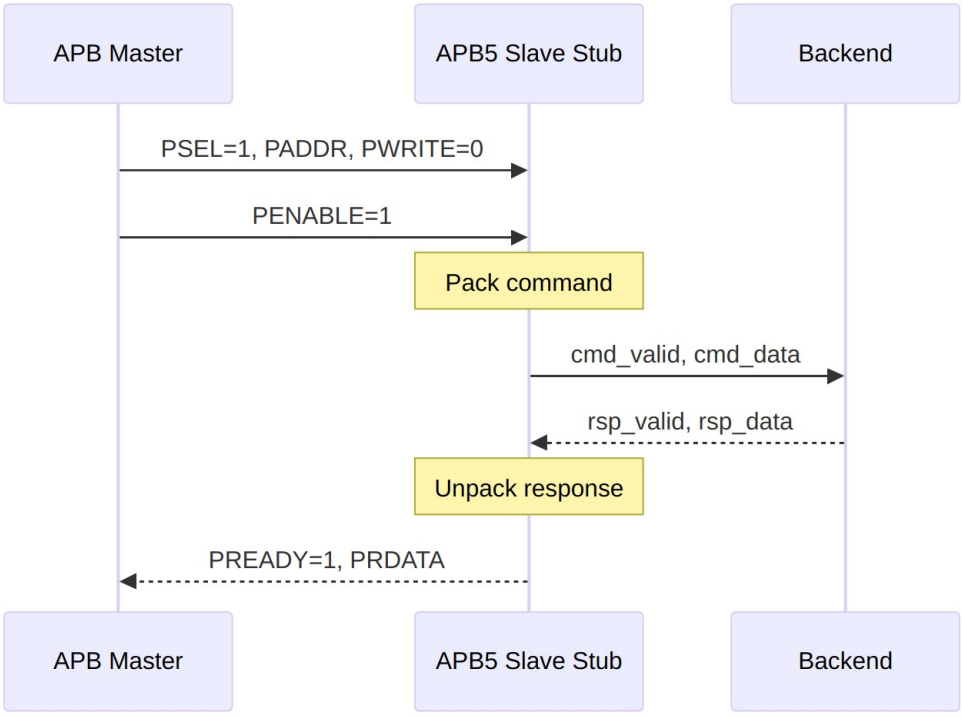
Field	Width	Description
pbuser	BUSER_WIDTH	Response user signal

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> XFER_DATA : PSEL & PENABLE & cmd_ready
    XFER_DATA --> IDLE : rsp_valid

    state IDLE {
        note right of IDLE : Wait for APB access
    }
    state XFER_DATA {
        note right of XFER_DATA : Wait for backend response
    }
```

8.4.3 State Descriptions

State	Description
IDLE	Waiting for APB transaction (PSEL & PENABLE)
XFER_DATA	Command sent, waiting for response from backend

8.5 Transaction Flow



Read Transaction

8.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- pclk
- APB signals (PSEL, PENABLE, PADDR, PWRITE, PREADY, PRDATA)
- cmd_valid, cmd_ready, cmd_data
- rsp_valid, rsp_ready, rsp_data
- State transitions IDLE -> XFER_DATA -> IDLE
```

8.6 Usage Example

```
apb5_slave_stub #(
    .DEPTH          (4),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
```

```
) u_apb5_slave_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PREADY   (s_apb_pready),
    .s_apb_PWAKEUP  (s_apb_pwakeup),
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Packed command interface
    .cmd_valid      (be_cmd_valid),
    .cmd_ready      (be_cmd_ready),
    .cmd_data       (be_cmd_data),

    // Packed response interface
    .rsp_valid      (be_rsp_valid),
    .rsp_ready      (be_rsp_ready),
    .rsp_data       (be_rsp_data),

    // Wake-up control
    .wakeup_request (be_wakeup)
);

// Simple backend: echo write data on reads
always_ff @(posedge apb_clk) begin
    if (!apb_rst_n) begin
        be_rsp_valid <= 1'b0;
    end else if (be_cmd_valid && be_cmd_ready) begin
        be_rsp_valid <= 1'b1;
        be_rsp_data <= {1'b0, stored_data, 8'h0};  // {pslverr,
prdata, pruser, pbuser}
    end else if (be_rsp_ready) begin
        be_rsp_valid <= 1'b0;
    end
end
```

8.7 Design Notes

8.7.1 Skid Buffers

The stub uses skid buffers on both command and response paths: - **Command skid buffer:** Absorbs command while FSM transitions - **Response skid buffer:** Allows backend to push response before slave ready

8.7.2 Parity Implementation

When `ENABLE_PARITY=1`: - **Checking:** Odd parity verified on incoming PWDATA, PADDR, control signals - **Generation:** Odd parity generated for outgoing PRDATA, PREADY, PSLVERR

8.7.3 Wake-up Handling

The `wakeup_request` input is registered before driving `s_apb_PWAKEUP`: - Provides clean synchronization - Single-cycle delay from request to PWAKEUP assertion

---

8.8 Related Documentation

- **APB5 Slave** - Full slave module
- **APB5 Master Stub** - Corresponding master stub

8.9 Navigation

- **<- Back to APB5 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

9 APB5 (Advanced Peripheral Bus - AMBA 5) Modules

Location: `rtl/amba/apb5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

9.1 Overview

The APB5 subsystem provides a complete implementation of the ARM AMBA 5 APB (Advanced Peripheral Bus) protocol, including masters, slaves, monitors, clock domain crossing, and testbench utilities.

APB5 extends APB4 with enhanced features for modern SoC designs while maintaining backward compatibility. The simple two-cycle handshake protocol is preserved, with additional signals for improved security, wake-up signaling, and error handling.

9.2 AMBA4 vs AMBA5 Comparison

Feature	APB4	APB5
Basic Protocol	Two-phase handshake	Two-phase handshake
Protection	PPROT[2:0]	PPROT[2:0] + PNSE
Wake-up	Not supported	PWAKEUP signal
User Signals	Not supported	PAUSER, PWUSER, PRUSER, PBUSER
Atomic Operations	Not supported	PEXCL, PEXOKAY
Error Response	PSLVERR	PSLVERR (enhanced semantics)

9.3 Module Categories

9.3.1 Core Protocol Components

Module	Description	Documentation	Status
apb5_master	Full-featured APB5 master with command/response interface	apb5_master.md	Documented
apb5_slave	Complete APB5 slave with buffered cmd/rsp interface	apb5_slave.md	Documented
apb5_slave_	APB5 slave with clock	apb5_slave_cdc.md	Documented

Module	Description	Documentation	Status
cdc	domain crossing support		
apb5_monitor	Transaction monitoring with 128-bit monitor bus + 64-bit timestamp	apb5_monitor.md	Documented

9.3.2 Clock-Gated Variants

Module	Description	Documentation	Status
apb5_master_cg	APB5 master with integrated clock gating	apb5_master_cg.md	Documented
apb5_slave_cg	APB5 slave with integrated clock gating	apb5_slave_cg.md	Documented
apb5_slave_cdc_cg	APB5 slave CDC with integrated clock gating	apb5_slave_cdc_cg.md	Documented

9.3.3 Testbench Utilities

Module	Description	Documentation	Status
apb5_master_stub	Lightweight APB5 master for testbench integration	apb5_master_stub.md	Documented
apb5_slave_stub	Lightweight APB5 slave for testbench integration	apb5_slave_stub.md	Documented

9.4 Key Features

9.4.1 APB5 Protocol Support

- **Full APB5 Compliance:** Complete AMBA 5 APB protocol implementation
- **PSTRB Support:** Byte-lane strobes for partial writes
- **PPROT Support:** Protection attributes for security-aware systems

- **PNSE Support:** Non-secure extension for TrustZone integration
- **PWAKEUP Support:** Low-power wake-up signaling
- **User Signals:** PAUSER, PWUSER, PRUSER, PBUSER for sideband data

9.4.2 Clock Domain Crossing

- **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains
- **Safe CDC:** Proper handshake-based clock domain crossing
- **Independent Frequencies:** Backend can run faster or slower than APB

9.4.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Signaling:** PWAKEUP for low-power state exit
- **Idle Detection:** Automatic clock gate when bus is idle

9.4.4 Monitoring and Debug

- **Transaction Monitoring:** Real-time protocol monitoring
- **64-bit Monitor Bus:** Standardized packet format
- **Error Detection:** Protocol violations, timeout detection
- **Performance Tracking:** Transaction counting, latency measurement

---

9.5 Quick Start

9.5.1 Using APB5 Master

```
apb5_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb5_master (
    .aclk           (clk),
    .aresetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),
```

```
    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),

    // APB5 master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
    .m_apb_PPROT    (pprot),
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr),
    .m_apb_PWAKEUP  (pwakeup)
);
```

9.5.2 Using APB5 Slave

```
apb5_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb5_slave (
    .aclk           (clk),
    .aresetn        (resetn),

    // APB5 slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),
    .s_apb_PWAKEUP  (pwakeup),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
```

```
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

9.6 Testing

All APB5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB5 tests
pytest val/amba/test_apb5*.py -v

# Run specific module tests
pytest val/amba/test_apb5_master.py -v
pytest val/amba/test_apb5_slave.py -v
pytest val/amba/test_apb5_slave_cdc.py -v
pytest val/amba/test_apb5_monitor.py -v
```

9.7 Protocol Details

9.7.1 APB5 Transfer Phases

APB5 uses the same two-phase protocol as APB4:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer
2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

9.7.2 APB5 Signal Descriptions

Signal	Direction	Description
PCLK	Input	APB clock
PRESETn	Input	Active-low reset
PSEL	Master to Slave	Slave select
PENABLE	Master to Slave	Enable (ACCESS phase indicator)
PREADY	Slave to Master	Transfer complete
PADDR	Master to Slave	Address bus
PWRITE	Master to Slave	Write enable (1=write, 0=read)
PWDATA	Master to Slave	Write data
PSTRB	Master to Slave	Byte lane strobes
PPROT	Master to Slave	Protection attributes
PNSE	Master to Slave	Non-secure extension (APB5)
PWAKEUP	Master to Slave	Wake-up signal (APB5)
PAUSER	Master to Slave	Address phase user signal (APB5)
PWUSER	Master to Slave	Write data user signal (APB5)
PRDATA	Slave to Master	Read data
PSLVERR	Slave to Master	Error response
PRUSER	Slave to Master	Read data user signal (APB5)
PBUSER	Slave to Master	Write response user signal (APB5)

9.8 Design Notes

9.8.1 Command/Response Architecture

All APB5 modules use a command/response interface pattern for backend integration, identical to the APB4 architecture:

Command Interface: - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes

Response Interface: - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag

9.8.2 Migration from APB4

APB5 modules are backward compatible with APB4 systems: - Connect APB5 signals to APB4 equivalents - Tie unused APB5 signals (PNSE, PWAKEUP, user signals) to default values - PNSE typically tied to 0 for secure mode - PWAKEUP typically tied to 0 for always-awake operation

---

9.9 Related Documentation

- **APB4 Modules** - APB4 protocol components
- **AXI5 Modules** - AXI5 protocol components
- **AXIS5 Modules** - AXI5-Stream components
- **GAXI Modules** - Generic AXI utilities

9.10 References

9.10.1 Specifications

- ARM AMBA 5 APB Protocol Specification
- ARM AMBA APB Protocol Specification v2.0 (APB4)

9.10.2 Source Code

- RTL: `rtl/amba/apb5/`
- Tests: `val/amba/test_apb5*.py`
- Framework: `bin/TBClasses/components/apb/`

Last Updated: 2025-12-26

9.11 Navigation

- **Back to RTLAmba Index**
- **Back to Main Documentation Index**